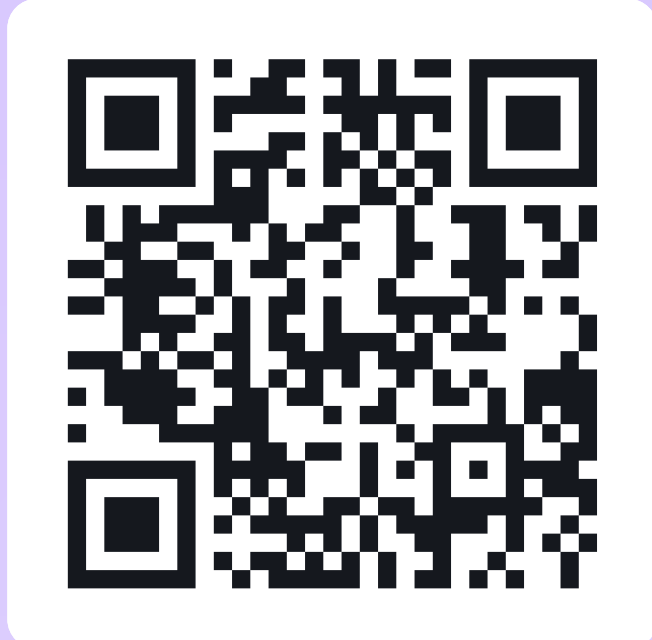


Evaluating and Mitigating the Misguidance Effect of Buggy Code in LLM-Generated Unit Tests

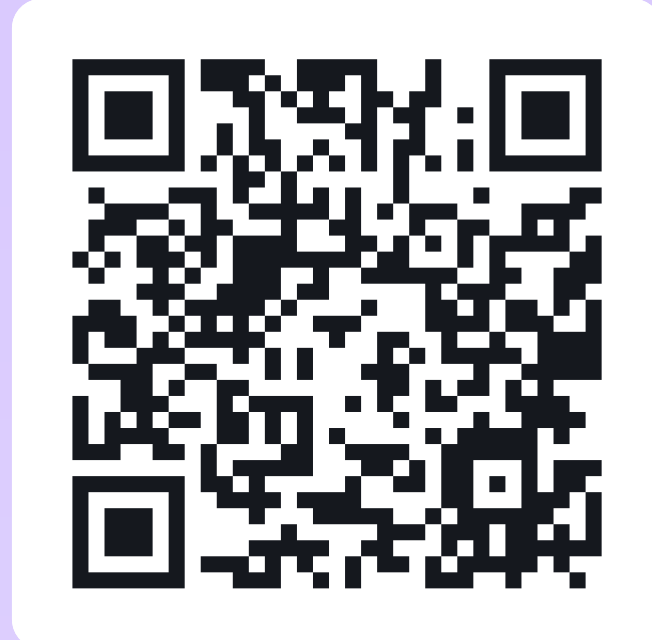
Junda Zhao¹ · Shurui Zhou² · Eldan Cohen¹
¹ Mechanical & Industrial Engineering
² Electrical & Computer Engineering — University of Toronto



UNIVERSITY OF
TORONTO



Paper
junda.zhao@mail.utoronto.ca
I am on the job market for postdoc positions.
Happy to chat, open to questions.



Replication package

Talk Fri 9 Oct · 14:40 · East Hall 2

The LLM generates a test that asserts the bug in the buggy input

Buggy method: StringUtils.equals (Defects4J Lang-14)

```
public static boolean equals(
    CharSequence cs1, CharSequence cs2) {
    if (cs1 == cs2)        return true;
    if (cs1 == null
        || cs2 == null)    return false;
    return cs1.equals(cs2);
}
```

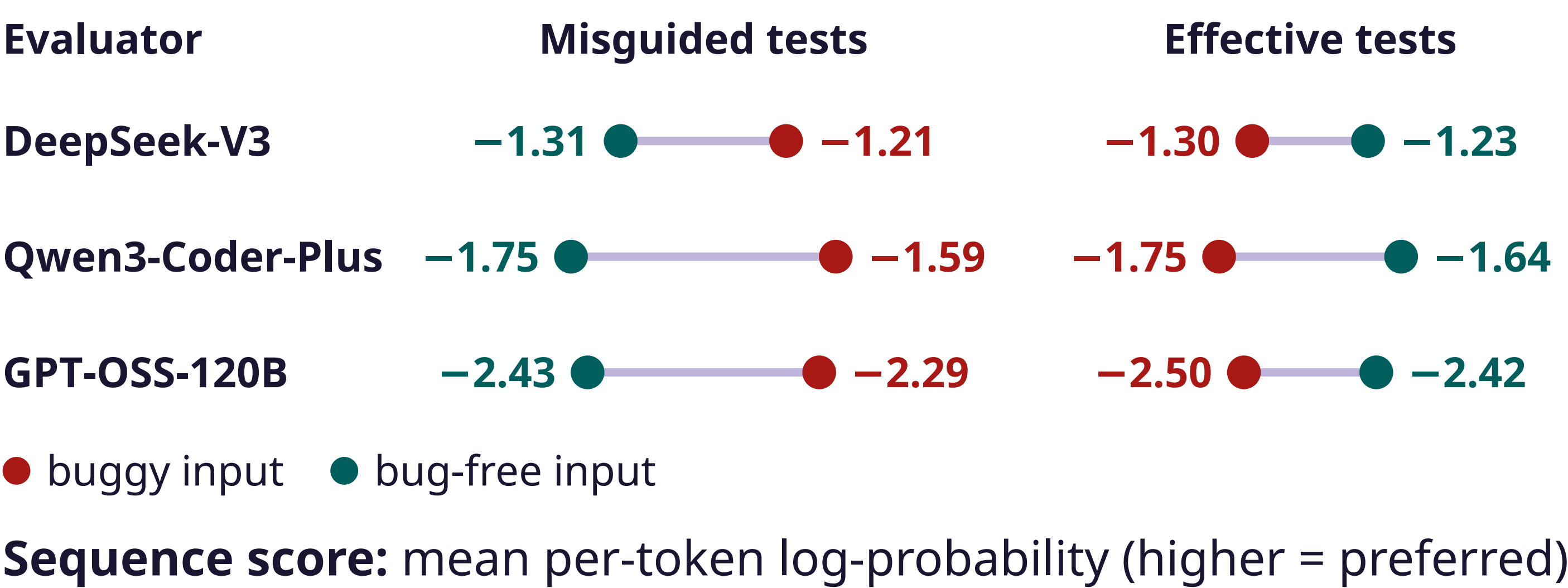
Test generated from the buggy method

```
assertFalse(StringUtils.equals("abc",
    new StringBuilder("abc")));

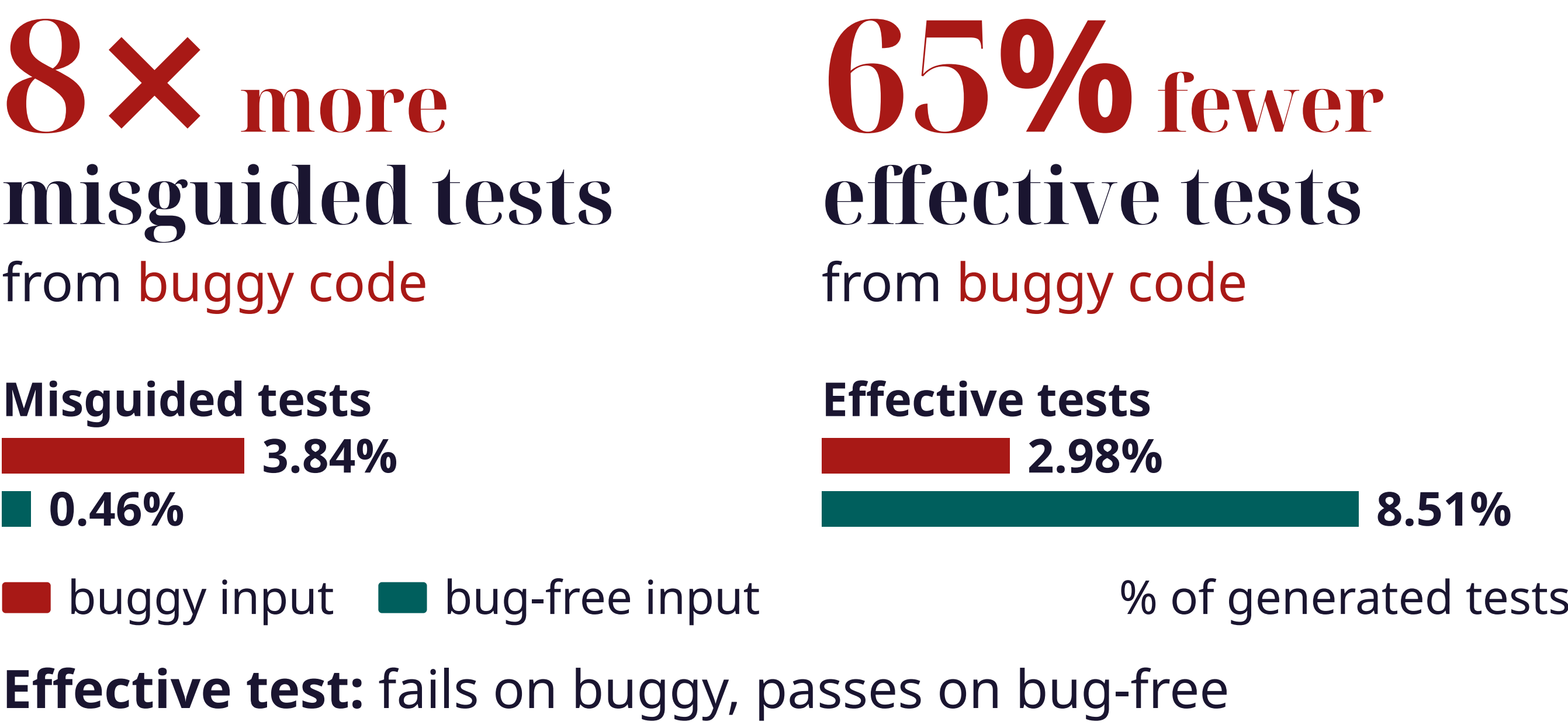
// the buggy body returns false here,
// since cs2 is a non-String object,
// and the LLM took that as intended.
```

Misguided test: ✓ passes on buggy ✗ fails on bug-free

Buggy code shifts the model's preference to misguided tests

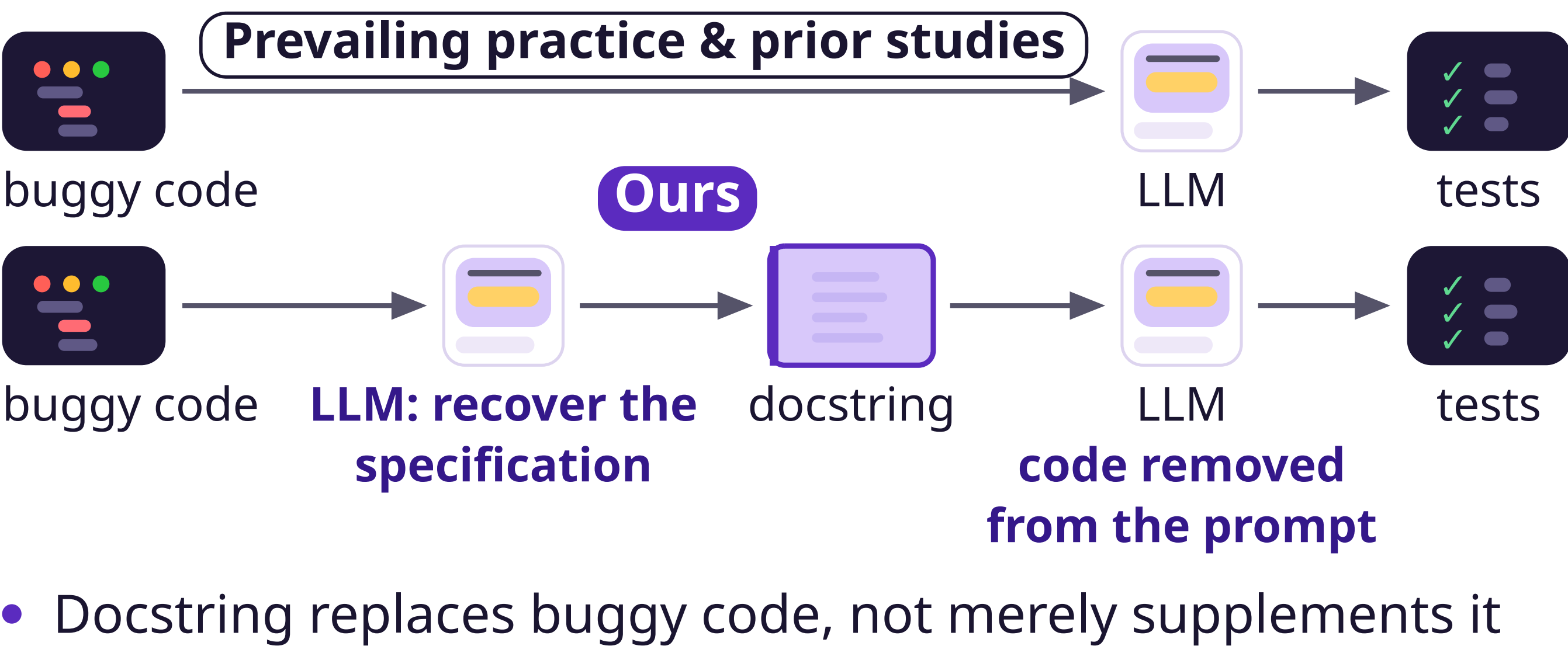


Buggy code misguides the LLM and reduces real bug detection

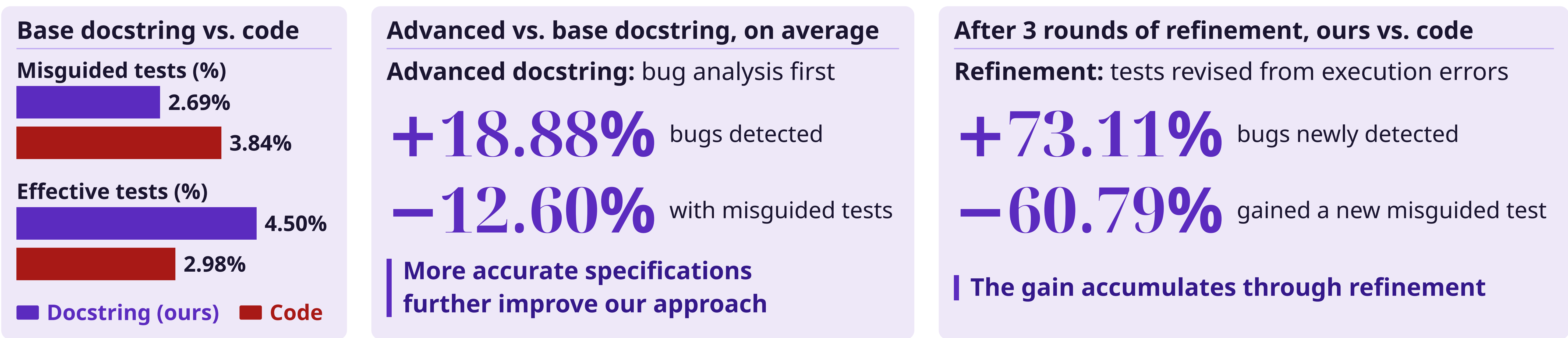


Evaluations on bug-free code likely overestimate real bug detection on buggy code

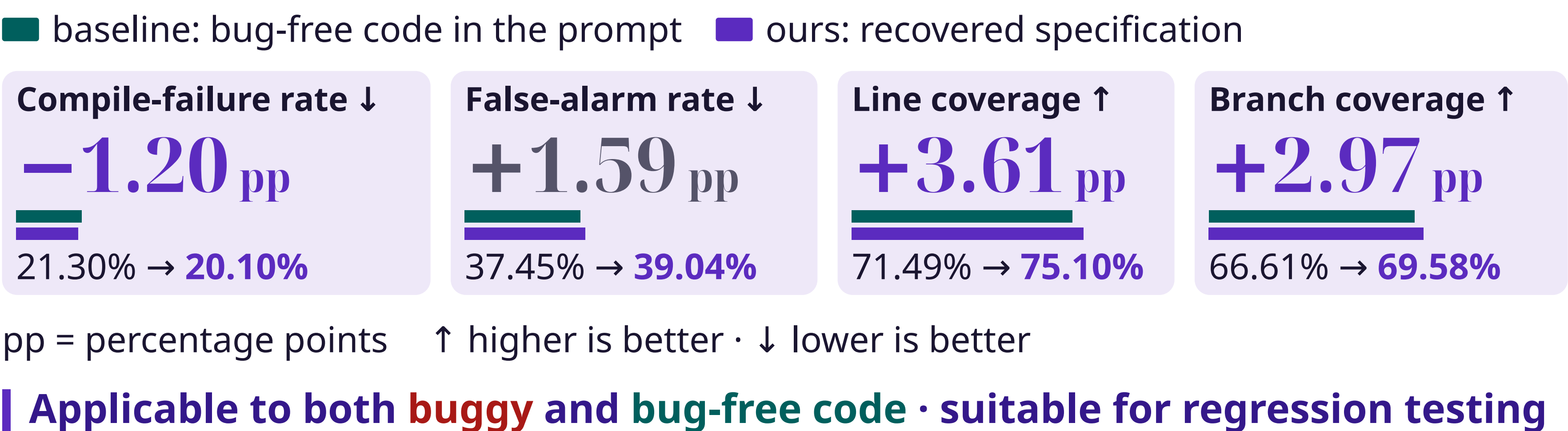
Our approach: specification-based testing LLM-generated docstring over source code



Our approach mitigates misguidance and detects more bugs



Little to no penalty on bug-free code



Future work

- More robust specification generation (e.g. multi-agent LLM reasoning with static program analysis)
- Formats beyond natural language: UML, formal specifications, requirements
- LLM-generated vs. human-written specifications: the approach's upper bound